Intermediate Python

Programming challenges and mini-projects

For students who know variables, `if` statements, and loops and want something more challenging.

No additional Python libraries are required: use any browser-based Python editor or your own installation. These four challenges are **independent**: choose what interests you. You are **not expected to finish all four** in today's session.

How to work

1. Read the toolbox, then try the **core task** yourself.
2. Run your code often. Test it on small examples and edge cases.
3. When it works, choose an extension or a different challenge.
4. If you use generative AI, **attempt a solution first**; then ask it to review or suggest an alternative. Test its suggestions too.

Suggested route: Number guessing → Caesar cipher → tic-tac-toe → penguin classifier. But you may start anywhere. The approximate levels are medium, medium, medium–hard, and medium–hard.

Challenge 1 — Guess the secret number

Goal. Build a game in which the computer picks a secret number between 1 and 100, and the player keeps guessing until they find it.

Toolbox: Random numbers, input, and repetition

`random.randint(1, 100)` chooses an integer from 1 to 100, including both endpoints. The function `input()` reads text, so use `int(...)` when you expect a number. A `while` loop repeats while its condition is true.

```
import random
secret = random.randint(1, 100)
guess = int(input("Your guess: "))

while guess != secret:
    # Give a hint, then read a new guess.
    ...
```

The `...` is a temporary placeholder; replace it with your code. Beware: forgetting to update the guess inside the loop can cause an infinite loop.

Core tasks

1. Print `Too low!` if the guess is below the secret number, or `Too high!` if it is above.
2. Ask for another guess until the number is found. Then print a congratulatory message.
3. Count the number of guesses and display the total at the end.

Try these tests

For testing, temporarily replace the random number with `secret = 42`. Check guesses `10`, `70`, `42`; the hints should be "Too low!", "Too high!", and then success after 3 guesses. Also test

guessing correctly on the first attempt.

Extensions (choose one or more)

A. Limit the player to seven guesses. What happens after the seventh wrong guess?

B. Handle invalid input without crashing (hint: `try`, `except ValueError`).

C. **Reverse the game:** the human thinks of a number from 1 to 100. Have the computer guess it by repeatedly choosing the midpoint of the remaining interval. How many guesses does it need at most? (This is *binary search*.)

Challenge 2 — The Caesar cipher

Goal. Write a function `encrypt(message, shift)` that shifts letters around the alphabet. For example, `HELLO` shifted by 3 becomes `KHOOR`.

Toolbox: Strings, functions, and wrapping around

A *function* packages reusable code. You can loop through a string one character at a time. `ord()` converts a character to its Unicode number and `chr()` converts a number back to a character. The remainder operator `%` helps wrap around the alphabet.

```
def encrypt(message, shift):
    result = ""
    for ch in message:
        # Add a transformed character to result.
        ...
    return result

print(ord("A"))  # 65
print(chr(65))    # A
print(27 % 26)   # 1
```

For an uppercase letter `ch`, its position from 0 to 25 is `ord(ch) - ord("A")`. To wrap around: `(position + shift) % 26`.

Core tasks

1. Implement `encrypt(message, shift)` for **uppercase A–Z**, including wrap-around from Z to A.
2. Preserve spaces and punctuation unchanged. For example, `HELLO, WORLD!` shifted by 3 becomes `KHOOR, ZRUOG!`.
3. Implement `decrypt(message, shift)`. Hint: a negative shift reverses encryption.

Try these tests

`encrypt("XYZ", 3)` → `"ABC"`; `encrypt("A B!", 1)` → `"B C!"`; decrypting your own ciphertext must recover the original message.

Extensions

A. Preserve lowercase letters too: `encrypt("Hello!", 3)` should yield `"Khoor!"`.

B. Given ciphertext but not the shift, try all 26 possible shifts. Which candidate looks like readable English?

C. Explain why a Caesar cipher should *not* be used for real security.

Challenge 3 — Tic-tac-toe: detect the winner

Goal. Given a finished or partially finished 3-by-3 board, decide whether X or O has three in a row. We are *not* building a full game yet.

Toolbox: Nested lists and functions

A list of lists represents a grid. Rows and columns are indexed starting at zero. A winner has three matching, nonempty entries in a row, column, or diagonal.

```
board = [
    ["X", "O", ""],
    ["X", "O", ""],
    ["X", "",  ""]
]
print(board[0][1])  # O: row 0, column 1

def winner(board):
    # Return "X", "O", or None.
    ...
```

Returning `None` means that no player has won yet. You may assume the board contains only `"X"`, `"O"`, and `""` (empty), and that at most one player has won.

Core tasks

1. Check all three rows for a winner.
2. Add checks for all three columns.
3. Add the two diagonals. Return `"X"` or `"O"` for a winner, otherwise `None`.

Try these tests

```
print(winner([["X","X","X"], ["O","","O"], ["","",""]]))  # X
print(winner([["O","X",""], ["O","X",""], ["O","",""]]))  # O
print(winner([["X","O",""], ["O","X",""], ["","","X"]]))  # X
print(winner([["X","O","X"], ["X","O","O"], ["O","X","X"]])) # None
```

Extensions

A. Write `game_status(board)` that returns `"X wins"`, `"O wins"`, `"Draw"`, or `"In progress"`.

B. Write `valid_move(board, row, col)` to check whether a move is on the board and its cell is empty.

C. Challenge: allow two humans to play in the terminal using `input()`. If you finish, can you design a simple computer opponent?

Challenge 4 — Penguins: a tiny classifier without libraries

Goal. Use a small dataset to explore how a program can make predictions from measurements. Everything uses ordinary Python lists, dictionaries, conditions, and loops.

Toolbox: Dictionaries, features, predictions, and accuracy

A *dictionary* stores labelled values. A *feature* is a measurement used to predict something. A *classifier* predicts a category; its *accuracy* is the fraction of predictions that match the true labels.

```
penguin = {"species": "Gentoo", "mass": 5000, "flipper": 217}
print(penguin["flipper"])  # 217

# Example rule, which you will evaluate and improve:
def predict_species(flipper):
    if flipper > 210:
        return "Gentoo"
    return "Other"
```

Important: In this small exercise, labels are `"Gentoo"` and `"Other"`. This is an illustrative toy dataset, not the full Palmer Penguins dataset. A good result here does not establish accuracy on real unseen penguins.

```
penguins = [
    {"species": "Other",  "mass": 3700, "flipper": 181},
    {"species": "Gentoo", "mass": 5000, "flipper": 217},
    {"species": "Other",  "mass": 3800, "flipper": 195},
    {"species": "Gentoo", "mass": 5500, "flipper": 225},
    {"species": "Other",  "mass": 3450, "flipper": 178},
    {"species": "Gentoo", "mass": 4800, "flipper": 212},
    {"species": "Other",  "mass": 4100, "flipper": 201},
    {"species": "Gentoo", "mass": 5100, "flipper": 209}
]
```

Core tasks

1. Calculate the average mass across all eight penguins. Find the heaviest penguin.
2. Calculate the average mass separately for `Gentoo` and `Other`.
3. Use the provided `predict_species(flipper)` rule for each penguin. Count correct predictions and calculate accuracy as a percentage.
4. Try thresholds of 200, 205, 210, and 215 mm. Which works best *on this tiny dataset*? Is that guaranteed to be best on future penguins?

Try these tests

The average mass is **about 4,431 g**; the heaviest is **5,500 g**. The default threshold of 210 mm makes **7 of 8 predictions correct**. (You should verify these by running your own code.)

Extensions

A. Write a reusable function `accuracy(data, threshold)`.

B. Split the penguins into a *training set* (for choosing the threshold) and a *test set* (for evaluating it). Why shouldn't you tune your rule on the test set?

C. Invent a rule using *both* flipper length and body mass. Compare its accuracy to the single-feature rule. Can your rules make mistakes on a different dataset?